

Balance de carga en un clúster de servidores mediante el método de Jacobi

Solución iterativa de sistemas de ecuaciones lineales

Estudiante: **Fernando Istaña Flores** Docente: **Jorge Luis Manrique Plasencia**
Escuela: **Ingeniería de Sistemas**

1. Planteamiento del problema

Un centro de datos cuenta con cuatro nodos interconectados que procesan peticiones web. La carga asignada a cada servidor, x_i (en miles de peticiones por minuto), depende de las peticiones de entrada directas y de la redistribución interna que realiza el balanceador entre los nodos conectados. El modelo en estado estacionario conduce al sistema $A \cdot x = b$:

$$\begin{aligned} 10x_1 - 2x_2 - x_3 &= 15 \\ -x_1 + 8x_2 - 2x_4 &= 18 \\ -2x_1 + 12x_3 - 3x_4 &= 25 \\ -x_2 - 2x_3 + 9x_4 &= 20 \end{aligned}$$

Se resuelve con el método de Jacobi partiendo de $x^{(0)} = [0, 0, 0, 0]^T$, con una tolerancia $\varepsilon = 10^{-4}$ sobre el error relativo.

2. Análisis preliminar: demostración de convergencia

Una matriz es **estrictamente diagonal dominante (EDD) por filas** cuando, en cada fila, el valor absoluto del elemento diagonal supera la suma de los valores absolutos de los demás elementos: $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$.

FILA	$ A_{ii} $	$\sum_{j \neq i} A_{ij} $	COMPARACIÓN	COCIENTE
1	10	$ -2 + -1 + 0 = 3$	$10 > 3 \checkmark$	$3/10 = 0.3000$
2	8	$ -1 + 0 + -2 = 3$	$8 > 3 \checkmark$	$3/8 = 0.3750$
3	12	$ -2 + 0 + -3 = 5$	$12 > 5 \checkmark$	$5/12 = 0.4167$
4	9	$0 + -1 + -2 = 3$	$9 > 3 \checkmark$	$3/9 = 0.3333$

Las cuatro filas cumplen la desigualdad, por lo tanto **A es estrictamente diagonal dominante por filas**.

Qué garantiza esta propiedad. Jacobi escribe el sistema como $x^{(k)} = T_J \cdot x^{(k-1)} + c$, con $T_J = -D^{-1}(L + U)$, donde D es la diagonal de A. La norma infinito de la matriz de iteración es el mayor cociente de la tabla: $\|T_J\|_\infty = \max(0.3000; 0.3750; 0.4167; 0.3333) = \mathbf{0.4167} < 1$. Como el radio espectral está acotado por cualquier norma, $\rho(T_J) \leq 0.4167 < 1$ (el valor exacto calculado es $\rho \approx 0.3574$). En consecuencia, el método de Jacobi **converge a la**

solución única del sistema para cualquier vector inicial, y el error se reduce en cada iteración al menos por el factor 0.4167, lo que anticipa una convergencia rápida.

3. Esquema iterativo

Despejando la variable de la diagonal de cada ecuación y usando únicamente los valores de la iteración anterior:

$$\begin{aligned}x_1^{(k)} &= (15 + 2x_2^{(k-1)} + x_3^{(k-1)}) / 10 \\x_2^{(k)} &= (18 + x_1^{(k-1)} + 2x_4^{(k-1)}) / 8 \\x_3^{(k)} &= (25 + 2x_1^{(k-1)} + 3x_4^{(k-1)}) / 12 \\x_4^{(k)} &= (20 + x_2^{(k-1)} + 2x_3^{(k-1)}) / 9\end{aligned}$$

Error relativo en cada iteración: $e^{(k)} = \|x^{(k)} - x^{(k-1)}\|_{\infty} / \|x^{(k)}\|_{\infty}$. El proceso se detiene cuando $e^{(k)} < 10^{-4}$.

Primera iteración, a mano: $x_1^{(1)} = 15/10 = 1.5$; $x_2^{(1)} = 18/8 = 2.25$; $x_3^{(1)} = 25/12 \approx 2.083333$; $x_4^{(1)} = 20/9 \approx 2.222222$, con error relativo 1 porque $x^{(0)} = 0$.

4. Código fuente documentado

```
"""
Métodos Numéricos · Unidad 2 · Sesión 7
Método de Jacobi – balance de carga en un clúster de 4 servidores.

x_i : peticiones asignadas al servidor i (miles por minuto)
Vector inicial x(0) = [0, 0, 0, 0]
Criterio de parada: error relativo ||x(k) - x(k-1)||inf / ||x(k)||inf < 1e-4

Fernando Istaña Flores · Universidad Peruana Unión
"""

def es_diagonal_dominante(A):
    """Verifica |a_ii| > sum_{j != i} |a_ij| en cada fila y devuelve el detalle."""
    detalle = []
    ok = True
    for i, fila in enumerate(A):
        diag = abs(fila[i])
        resto = sum(abs(v) for j, v in enumerate(fila) if j != i)
        cumple = diag > resto
        ok = ok and cumple
        detalle.append((i + 1, diag, resto, cumple))
    return ok, detalle

def norma_jacobi(A):
    """||T_J||inf = max_i sum_{j != i} |a_ij| / |a_ii| (cota de convergencia)."""
    return max(sum(abs(v) for j, v in enumerate(f) if j != i) / abs(f[i])
               for i, f in enumerate(A))

def jacobi(A, b, x0, tol=1e-4, max_iter=100):
    """
    Método de Jacobi. Cada componente nueva usa SOLO valores de la iteración anterior:
    """
```

```

        x_i(k) = ( b_i - sum_{j != i} a_ij * x_j(k-1) ) / a_ii
Devuelve la solución y el historial [(k, x(k), error)].
"""
n = len(b)
x_ant = list(x0)
historial = [(0, list(x0), None)]
for k in range(1, max_iter + 1):
    x_nuevo = [0.0] * n
    for i in range(n):
        suma = sum(A[i][j] * x_ant[j] for j in range(n) if j != i)
        x_nuevo[i] = (b[i] - suma) / A[i][i]
    dif = max(abs(x_nuevo[i] - x_ant[i]) for i in range(n))
    norma = max(abs(v) for v in x_nuevo)
    error = dif / norma if norma != 0 else dif
    historial.append((k, x_nuevo, error))
    if error < tol:
        return x_nuevo, historial
    x_ant = x_nuevo
return x_ant, historial

if __name__ == "__main__":
    A = [[10, -2, -1, 0],
          [-1, 8, 0, -2],
          [-2, 0, 12, -3],
          [0, -1, -2, 9]]
    b = [15, 18, 25, 20]
    x0 = [0, 0, 0, 0]

    print("=" * 78)
    print("ANÁLISIS PRELIMINAR – ¿A es estrictamente diagonal dominante por filas?")
    print("=" * 78)
    ok, det = es_diagonal_dominante(A)
    for fila, diag, resto, cumple in det:
        print(" Fila %d: |a_ii| = %2d > suma resto = %d %s"
              % (fila, diag, resto, "cumple" if cumple else "NO cumple"))
    print(" Resultado: A %s EDD" % ("ES" if ok else "NO es"))
    print(" ||T_J||inf = %.4f < 1 -> Jacobi converge para cualquier x(0)" % norma_jacobi(A))

    print()
    print("=" * 78)
    print("ITERACIONES DE JACOBI (tol = 1e-4, error relativo en norma infinito)")
    print("=" * 78)
    x, hist = jacobi(A, b, x0, tol=1e-4)
    print(" k          x1          x2          x3          x4          error")
    for k, xk, err in hist:
        e = "      -" if err is None else "%.6e" % err
        print("%3d %10.6f %10.6f %10.6f %10.6f %s" % (k, xk[0], xk[1], xk[2], xk[3], e))

    print()
    print("Solución aproximada tras %d iteraciones:" % (len(hist) - 1))
    for i, v in enumerate(x):
        print(" x%d = %.6f miles de peticiones/min" % (i + 1, v))
    r = [sum(A[i][j] * x[j] for j in range(4)) - b[i] for i in range(4)]
    print(" residuo A·x - b = (" + ", ".join("%.2e" % v for v in r) + ")")

```

5. Salida del programa

```

=====
ANÁLISIS PRELIMINAR – ¿A es estrictamente diagonal dominante por filas?
=====

```

```

Fila 1: |a_ii| = 10 > suma resto = 3 cumple
Fila 2: |a_ii| = 8 > suma resto = 3 cumple
Fila 3: |a_ii| = 12 > suma resto = 5 cumple
Fila 4: |a_ii| = 9 > suma resto = 3 cumple
Resultado: A ES EDD
||T_J||inf = 0.4167 < 1 -> Jacobi converge para cualquier x(0)

```

```

=====
ITERACIONES DE JACOBI (tol = 1e-4, error relativo en norma infinito)
=====

```

k	x1	x2	x3	x4	error
0	0.000000	0.000000	0.000000	0.000000	—
1	1.500000	2.250000	2.083333	2.222222	1.000000e+00
2	2.158333	2.993056	2.888889	2.935185	2.691415e-01
3	2.387500	3.253588	3.176852	3.196759	8.850628e-02
4	2.468403	3.347627	3.280440	3.289699	3.094370e-02
5	2.497569	3.380975	3.317159	3.323167	1.086040e-02
6	2.507911	3.392988	3.330387	3.335032	3.898688e-03
7	2.511636	3.397247	3.335077	3.339307	1.380480e-03
8	2.512957	3.398781	3.336766	3.340822	4.970880e-04
9	2.513433	3.399325	3.337365	3.341368	1.762051e-04
10	2.513602	3.399521	3.337581	3.341562	6.347358e-05

Solución aproximada tras 10 iteraciones:
x1 = 2.513602 miles de peticiones/min
x2 = 3.399521 miles de peticiones/min
x3 = 3.337581 miles de peticiones/min
x4 = 3.341562 miles de peticiones/min
residuo $A \cdot x - b = (-6.08e-04, -5.56e-04, -9.18e-04, -6.28e-04)$

6. Historial de iteraciones

K	X ₁	X ₂	X ₃	X ₄	ERROR RELATIVO
0	0.000000	0.000000	0.000000	0.000000	—
1	1.500000	2.250000	2.083333	2.222222	1.000e+00
2	2.158333	2.993056	2.888889	2.935185	2.691e-01
3	2.387500	3.253588	3.176852	3.196759	8.851e-02
4	2.468403	3.347627	3.280440	3.289699	3.094e-02
5	2.497569	3.380975	3.317159	3.323167	1.086e-02
6	2.507911	3.392988	3.330387	3.335032	3.899e-03
7	2.511636	3.397247	3.335077	3.339307	1.380e-03
8	2.512957	3.398781	3.336766	3.340822	4.971e-04
9	2.513433	3.399325	3.337365	3.341368	1.762e-04
10	2.513602	3.399521	3.337581	3.341562	6.347e-05

Solución aproximada (k = 10, error $6.35 \times 10^{-5} < 10^{-4}$): $x \approx (2.5136; 3.3995; 3.3376; 3.3416)^T$ miles de peticiones por minuto. La solución exacta por eliminación es

(2.513696; 3.399629; 3.337700; 3.341670), de modo que el error absoluto máximo de la aproximación es del orden de 1.2×10^{-4} .

7. Análisis de resultados

Convergencia. El error relativo cae aproximadamente a un tercio en cada iteración ($1 \rightarrow 0.269 \rightarrow 0.0885 \rightarrow 0.0309 \rightarrow \dots$), comportamiento coherente con el radio espectral $\rho(T_J) \approx 0.357$ obtenido en el análisis preliminar. Bastaron 10 iteraciones para alcanzar la tolerancia exigida, sin ninguna oscilación, lo que confirma en la práctica lo que la dominancia diagonal garantizaba en teoría.

Interpretación en el clúster. En estado estacionario, el balanceador asigna alrededor de 2 514 peticiones por minuto al servidor 1 y entre 3 338 y 3 400 a los servidores 2, 3 y 4, que quedan prácticamente equilibrados, con una diferencia menor al 2 % entre ellos. El servidor 1 es el que opera con menos carga, por lo que sería el candidato natural para absorber picos de tráfico. Todas las cargas resultan positivas y físicamente coherentes.

Ventaja del método iterativo. Cada iteración de Jacobi calcula las cuatro componentes de forma independiente a partir del vector anterior, por lo que puede paralelizarse: en un clúster real, cada nodo podría actualizar su propia carga con los valores que publicaron sus vecinos en el ciclo previo.

8. Conclusiones

La matriz del sistema es estrictamente diagonal dominante por filas, con $\|T_J\|_\infty = 0.4167 < 1$, lo que garantiza la convergencia del método de Jacobi desde cualquier vector inicial. Partiendo de $x^{(0)} = 0$, el método alcanzó la tolerancia $\varepsilon = 10^{-4}$ en 10 iteraciones y entregó $x \approx (2.5136; 3.3995; 3.3376; 3.3416)^T$, que coincide con la solución exacta hasta la cuarta cifra decimal. El resultado describe un clúster balanceado en el que los servidores 2, 3 y 4 soportan cargas casi iguales y el servidor 1 opera con una carga menor.